40% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Important Notice

The percentage shown reflects a probabilistic assessment and should be treated as an indicator, not a definitive conclusion. Results may vary depending on writing style, subject matter, and document structure. Always use this report as one of several points of reference rather than a sole determining factor.

Guidelines

This report analyzes long-form writing, such as essays, dissertations, and articles. Highlight colors indicate likelihood of AI detection flagging: cyan for high, light purple for moderate, unhighlighted for low.

Non-qualifying content (e.g. bullet points, annotated bibliographies, tables) is not analyzed and may cause the highlighted text to differ from the overall percentage shown. If the document contains non-selectable text, scanned images, or other formatting issues, the score may be inaccurate.

Details

Word Count: 1,418

Document Size: 687.8 KB

Score Interpretation

0% - 19%	Low likelihood of AI-generated content
20% - 50%	Moderate likelihood of AI-generated content
51% - 100%	High likelihood of AI-generated content

Highlight Legend

- High likelihood of being Flagged
- Moderate likelihood of being flagged
- Low likelihood of being flagged

Report generated: March 26, 2026 at 03:35 AM UTC

FACULTY OF ENGINEERING, BUILT ENVIRONMENT AND INFORMATION TECHNOLOGY

BACHELOR IN COMPUTER SCIENCE (HONOURS) (ODL AND WEEKEND PROGRAMME)

BCL1144 PROGRAMMING FUNDAMENTALS

CONTINUOUS ASSESSMENT (ASSIGNMENT 30%): OOP PROGRAM

APPOINTMENT BOOKING SYSTEM

DUE DATE SUBMISSION: 25 MARCH 2026, 11:59PM

STUDENT NAME	CHAN JING YI
STUDENT ID	SUOL2500321
LECTURER'S NAME	NOR ANIS ASMA BINTI SULAIMAN

DECLARATION OF INTEGRITY:

I hereby declare that this lab assessment submission is my own independent work and does not contain plagiarized content, unauthorized assistance, or any form of academic dishonesty. I confirm that I have adhered to the academic integrity policies outlined by the university.

I understand that if any form of academic dishonesty, including plagiarism, falsification of results, or unauthorized collaboration, is detected in this submission, I may face disciplinary actions. This may include, but is not limited to, receiving a failing grade for this assessment or further academic penalties as determined by the university's academic integrity committee.

By signing below, I acknowledge my understanding and agreement with this declaration.

Student's Signature: CHAN JING YI_____ Date: 25/3/26

Marking Scheme

Criteria	CLO	Extended Abstract (Highest Level)	Relational	Multi-structural	Uni-structural	Pre-structural	Weight (Marks)	Mark Allocation
OOP Structure	CLO2	(28–30) Sophisticated, well-integrated class relationships; meaningful abstraction; extensible design	(23–27) Classes logically integrated; proper use of inheritance/interface	(17–22) Multiple classes present but weak integration	(10–16) Basic class creation with limited relationships	(0–9) Incorrect or missing structure	30	
Core Functionality	CLO2	(18–20) All features robust, efficient, logically implemented	(14–17) All required functions working coherently	(10–13) Most features work independently	(5–9) Only basic feature works	(0–4) Program not functioning	20	
File Handling	CLO2	(14–15) Structured data persistence with robust exception handling	(11–13) Correct read/write with appropriate error handling	(7–10) Basic save/load works	(3–6) Partial implementation	(0–2) No meaningful file handling	15	
GUI Implementation	CLO2	(14–15) Intuitive layout, coherent interaction flow	(11–13) Functional interface with logical layout	(7–10) Basic components present	(3–6) Minimal interface	(0–2) No GUI	15	
Code Quality	CLO2	(9–10) Modular, reusable, clean, well-documented	(7–8) Structured and readable	(5–6) Understandable but limited refinement	(2–4) Poor organisation	(0–1) Unreadable/disorganised	10	

Documentation & Professional Practice	CLO3	(9–10) Insightful reflection; strong ethical awareness; clear improvement discussion	(7–8) Clear explanation; integrity declaration included	(5–6) Basic explanation provided	(2–4) Minimal documentation	(0–1) No documentation or integrity issue	10	
TOTAL							**100 Marks**	

CLO ALIGNMENT SUMMARY

CLO	**Total Marks**	**Percentage**
CLO2	90	90%
CLO3	10	10%

PERFORMANCE INTERPRETATION SCALE

Overall Mark	**Performance Category**
80–100	Excellent (Relational to Extended Abstract)
65–79	Good (Relational level achieved)
50–64	Satisfactory (multi-structural level)
40–49	Weak (Uni-structural level)
<40	Unsatisfactory (pre-structural level)

Overall Feedback

First Marker:

_____
_____
_____
_____

Total Mark: _____

Signature	Date

Second Marker:

_____
_____
_____
_____

Total Mark: _____

Signature	Date

Table of Contents

1. PROBLEM DESCRIPTION **9**

2. CLASS DIAGRAM **9**

3. EXPLANATION OF OOP IMPLEMENTATION **10**

3.1 ABSTRACTION AND ENCAPSULATION 10
3.2 INHERITANCE 10
3.3 POLYMORPHISM 10

4. EXPLANATION OF FILE HANDLING **10**

5. REFLECTION **11**

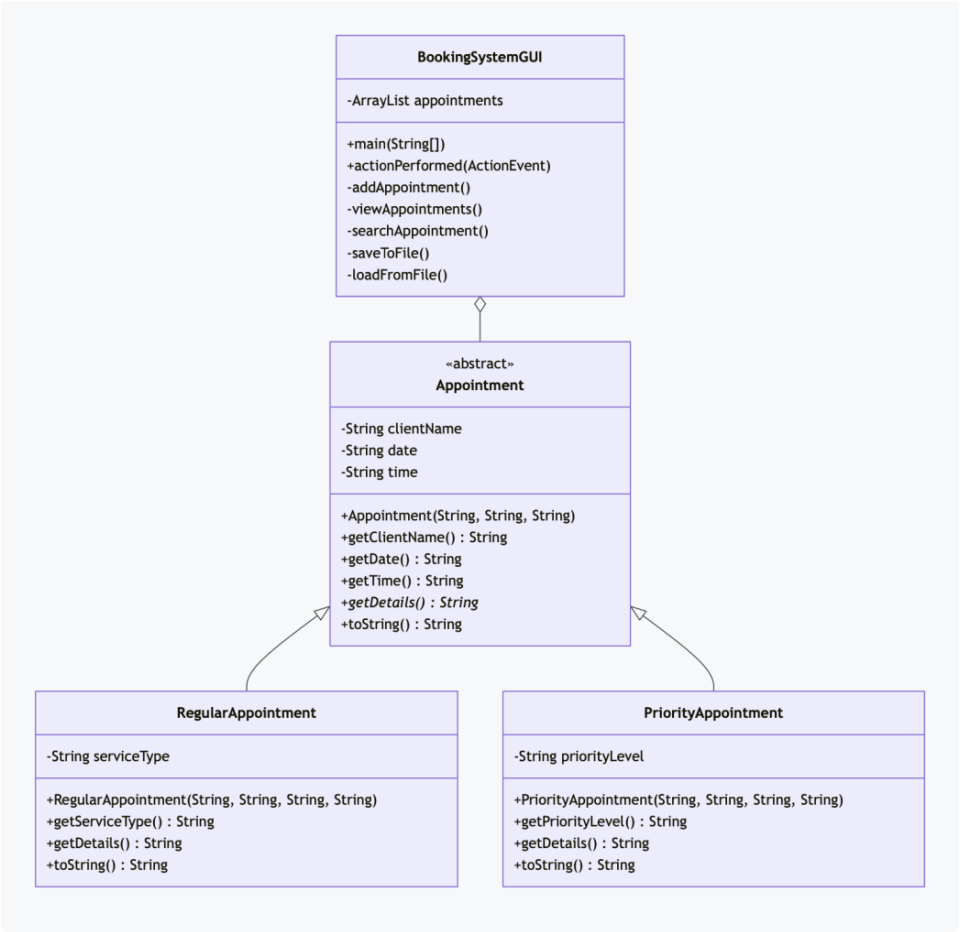
APPENDIX **12**

1. Problem Description

In contemporary service-oriented industries, the efficient management of client schedules is paramount to operational success. Manual booking processes are often prone to errors, such as double bookings or misplaced records, which can lead to significant administrative overhead and client dissatisfaction. To address these challenges, this project introduces a “Appointment Booking System” developed in Java. The system provides a centralized digital interface for recording, viewing, and searching for appointments. The system provides for advanced scheduling by dividing appointments into “Regular” and “Priority” kinds, which prioritizes urgent client requirements while keeping a continuous flow for normal services. Furthermore, the integration of persistent file storage ensures that data is preserved across application sessions, providing a reliable alternative to complex database systems for small-scale applications.

2. Class Diagram

The following class diagram illustrates the architectural structure of the system, highlighting the relationships between the abstract base class and its specialized subclasses.



3. Explanation of OOP Implementation

The system leverages four core principles of Object-Oriented Programming (OOP) to ensure a modular, extensible, and maintainable codebase.

3.1 Abstraction and Encapsulation

The Appointment class is defined as an abstract class, serving as a blueprint for all specific appointment types. It encapsulates shared attributes such as clientName , date , and time using the private access modifier. Public getter and setter methods are used to tightly restrict access to certain fields, guaranteeing data integrity and preventing unwanted external alteration. The abstract method getDetails() enforces a contract that all subclasses must implement their own specialized reporting logic.

3.2 Inheritance

Inheritance is demonstrated through the RegularAppointment and PriorityAppointment classes, both of which extend the Appointment base class. This structure promotes code reuse by allowing subclasses to inherit the foundational attributes and methods of the parent class while introducing specialized fields, such as serviceType for regular bookings and priorityLevel for urgent ones. This hierarchical design simplifies the management of diverse appointment categories.

3.3 Polymorphism

Polymorphism is achieved through overriding. Both subclasses provide their own unique implementation of the getDetails() and toString() methods. In the BookingSystemGUI class, an ArrayList<Appointment> is used to store objects of both subclasses.

During the “View All” operation, the system iterates through this list and invokes the overridden methods. The Java Virtual Machine (JVM) dynamically determines the correct implementation to execute based on the actual object type at runtime, allowing the system to handle different appointment types uniformly.

4. Explanation of File Handling

Data persistence is managed through a text-based file system using the java.io package. The system implements two primary operations: saveToFile() and loadFromFile() . For data exportation, the PrintWriter class is utilized to write appointment details into a comma-separated values (CSV) format within a file named appointments.txt . This format guarantees that the data is easily accessible and comprehensible by humans. For data importation, the Scanner class reads the file line-by-line, splitting the strings into individual attributes to reconstruct the Appointment objects within the application's memory.

Robust error handling is implemented using try-catch blocks. The saveToFile() method catches IOException to handle potential write errors, while loadFromFile() specifically addresses FileNotFoundException and general Exception types to prevent the application from crashing if the

data file is missing or corrupted. This approach ensures a professional user experience by providing informative feedback via the GUI's display area.

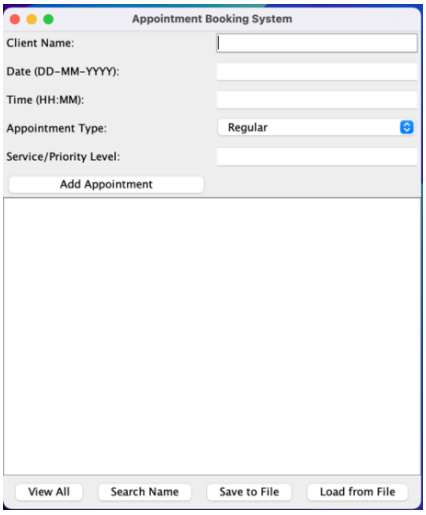
5. Reflection

Developing the Appointment Booking System was a comprehensive exercise in applying theoretical programming concepts to a functional software solution. The transition from console-based applications to a Graphical User Interface (GUI) using Java Swing presented the most significant learning curve. Managing the layout managers, such as BorderLayout and GridLayout , required a meticulous approach to ensure the interface remained intuitive and visually organized. One of the primary challenges encountered during development was the implementation of polymorphic file handling. Since the ArrayList contains different types of appointment objects, the saveToFile method had to utilize the instanceof operator to correctly identify and extract specialized fields like serviceType or priorityLevel . This highlighted the practical importance of understanding object types within a class hierarchy. Similarly, during the loading phase, the system had to parse a specific string identifier (“Regular” or “Priority”) to decide which subclass to instantiate. This “factory-like” logic was crucial for maintaining the integrity of the specialized data.

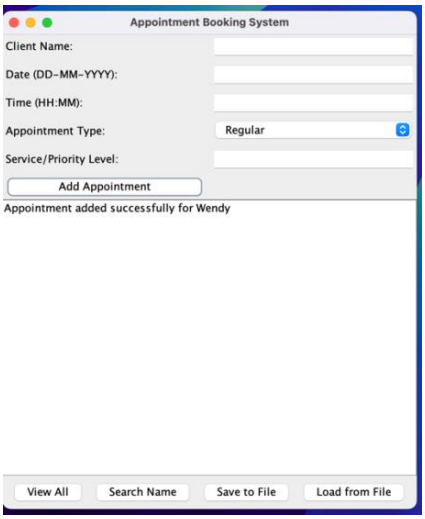
A significant limitation of the current prototype is its reliance on a simple text file for storage. While effective for small datasets, a text file lacks the advanced querying and indexing capabilities of a relational database. For instance, searching for a specific client name requires a linear search through the entire list, which would become inefficient as the number of records grows into the thousands. Future iterations of this project could integrate a database like SQLite or MySQL to enhance performance and data security. Furthermore, the GUI could be improved by incorporating data validation logic. Currently, the system accepts any string input for dates and times. Implementing a JDatePicker or regex-based validation for the DD-MM- YYYY format would significantly reduce user errors and improve the reliability of the system. Overall, this project has deepened my understanding of the OOP lifecycle, from design (Class Diagrams) to implementation and robust error management.

Appendix

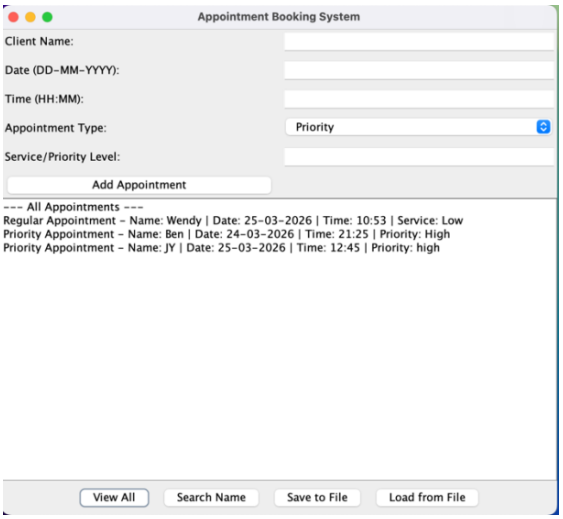
GUI



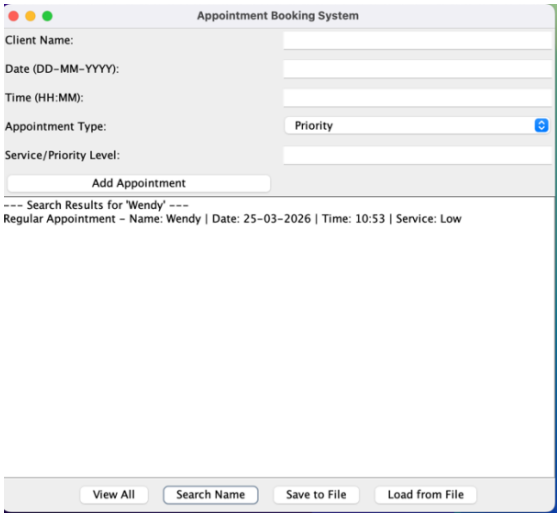
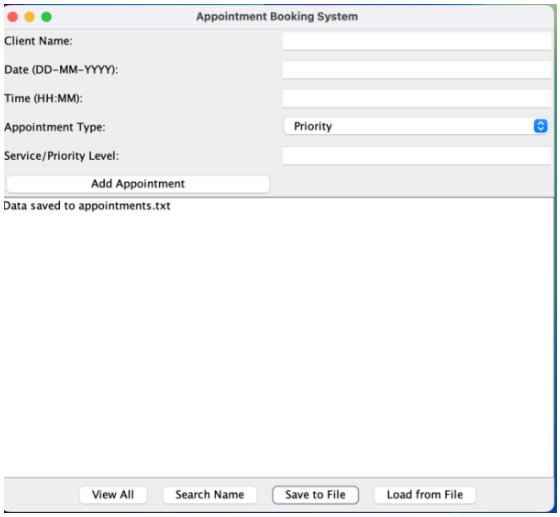
Click “Add Appointment” after filled in all the fields



Click “View All”



After clicked “Search Name”



Data save to appointments.txt after clicked “Save to File”